Webhook hardening report

> **Sample deliverable (Standard).** The client, "LedgerLark" (a fictional invoicing tool), and its integration are invented for this sample. It audits an integration against the seven rules this kit implements and lists the fixes. The "where the kit does it" column links to real code and tests in this repository.

1. Scope

One Express endpoint (`POST /stripe/webhook`) handling `checkout.session.completed`, `customer.subscription.*` and `invoice.*`, writing to a Postgres `accounts` table. Stripe **test mode** only.

2. Findings

#	Rule	Finding in the audited integration	Severity	Fix	Where the kit does it
1	Read the **raw body**, verify the signature before parsing	`express.json()` is mounted globally, so the handler verifies a re-serialised body; verification fails intermittently and someone "fixed" it by skipping verification in staging	High	Mount `express.raw({type: "application/json"})` on this route only; verify with the endpoint secret; reject with 400	`nextjs/src/lib/webhook/ingest.ts`; chaos scenario **Tampered payload**
2	Idempotent on `event.id`	Each delivery re-runs side effects	High	`INSERT ... ON CONFLICT (event_id) DO NOTHING`; treat a conflict as "already accepted"	`ingest.ts`; chaos scenario **Duplicate delivery**
3	Insert and enqueue atomically	Event is handled inline; a crash after the DB write but before the response loses the retry semantics	Medium	Insert the event row and the job in one transaction; return 200 fast; process in a worker with retries	`nextjs/src/lib/queue.ts`, `ingest.test.ts` ("enqueue failure rolls back")

#	Rule	Finding in the audited integration	Severity	Fix	Where the kit does it
4	Don't trust event order	Handler applies the event payload as truth; an old `updated` overwrites newer state	High	Fetch the latest object from the Stripe API; apply only if not older (compare-and-set); take a per-subscription lock	`nextjs/src/worker/process-event.ts`; chaos scenario **Out-of-order**
5	One function decides access	Access is computed in three places with different rules (`status === 'active'` in one, period end in another)	High	A single pure `deriveEntitlement`; access checks read only the entitlements table	`nextjs/src/lib/entitlements/derive.ts`, `docs/state-machine.md`
6	Never trust the success redirect	The checkout success page sets `plan = pro` from the `session_id` in the URL	High	Success page only reports server state (poll); the webhook worker grants access	`nextjs/src/app/checkout/success/`
7	Repair missed events	No reconciliation: a webhook that never arrives leaves a paying customer without access until they contact support	Medium	Scheduled reconcile that lists subscriptions from Stripe and repairs drift, with a report	`nextjs/src/reconcile/reconcile.ts`; chaos scenario **Missed webhook**

Also noted: the endpoint secret was committed in an example `.env` (rotate it; keep secrets out of the repo), and the handler used a live-mode key in staging (test mode only until sign-off).

3. Fix plan and order

1. Rotate the endpoint secret and remove secrets from the repo (same day).
2. Fix raw-body verification (finding 1), then idempotency and atomic enqueue (2, 3).
3. Introduce `deriveEntitlement` and remove the redirect grant (5, 6).
4. Add order handling (4), then reconciliation (7).

4. Tests to keep

Tampered/re-indented/unsigned payloads are rejected; a duplicate delivery has one effect; an enqueue failure returns 500 with no row; an older event cannot overwrite a newer one; a subscription with an unknown price fails loudly instead of granting access. Equivalent tests exist for both reference implementations.

5. Evidence that the mechanisms work

Live results from the kit's failure-injection scenarios against Stripe test mode, generated from `chaos/results/`:

Scenario	`nextjs`	`spring-boot`
Duplicate delivery	PASS	PASS
Out-of-order delivery (updated before created)	PASS	PASS
Missed webhook (listener down while the customer pays)	PASS	PASS
Tampered payload	PASS	PASS
Renewal failure (test clock + failing card)	PASS*	PASS* (latest of 3 runs; 2 FAIL)

`PASS*` = every check passed, but the account's Stripe retry setting ended the subscription as `canceled` instead of `unpaid` (access is removed either way). A cell that mentions FAIL runs shows the latest run and how many recorded runs failed; see `docs/chaos.md` for every check.